

How and Why to Provide Input Data to Vanadis 3D v3.1

Guide to tailoring $Q(t)$, $P(S)$, $v(x,y,z,t)$, $K(x,y,z,t)$, $\alpha(x,y,z,t)$, and other variable fields

Vanadis is intentionally structured so that the FEM core, DR stabilization, quadrature, and solver do not need to be redesigned whenever the physical description changes. Several small routines act as the physics/input layer: `source_term`, `licz_p`, `velocity_field`, `diffusion_field`, and `alpha_field`. Time dependence, spatial dependence, measurements, meteorological fields, or external CFD data should normally be introduced there.

Design rule: change the data/physics layer, not the FEM assembly. A field routine should return the physical value at the requested point (x,y,z,t) ; Vanadis then uses it at the proper Gauss points, at both time levels, and inside DR.

1. Model equation and where the data enter

A convenient form of the transport equation in the current version is:

$$\partial S / \partial t + v(x,y,z,t) \cdot \nabla S - \nabla \cdot (K(x,y,z,t) \nabla S) + P(S, \dots) S = Q(x,y,z,t)$$

On the Robin/deposition surface, the boundary condition is of the form:

$$-K \nabla S \cdot n = \alpha(x,y,z,t) S - q_s(x,y,z,t)$$

Quantity	SI unit	Current input hook	Meaning
S	kg/m ³ internally	FEM solution	concentration; converted to mg/m ³ only when writing results
Q	kg/s per source element	<code>source_term</code>	total emission assigned to the element
P	1/s	<code>licz_p</code>	removal/decay coefficient; physical term is $P(S) \cdot S$
v	m/s	<code>velocity_field</code>	advective velocity field
K = (Kx,Ky,Kz)	m ² /s	<code>diffusion_field</code>	effective diffusion/dispersion, currently diagonal
α	m/s	<code>alpha_field</code>	Robin/deposition transfer coefficient
q_s	kg/(m ² ·s)	<code>alpha_field</code>	optional surface source/flux
t, x,y,z	s, m	field-routine arguments	time and physical Gauss-point coordinates

Keep all physics in SI units. Do not provide P in 1/h, K in cm²/s, or concentration in mg/m³ internally without explicit conversion. The solver assumes consistent SI; concentration conversion to mg/m³ belongs only at output.

2. Levels of input-data complexity

Not every simulation needs an external data file. Use the simplest representation that captures the relevant physics. This reduces error sources and makes validation easier.

Level	When to use it	Implementation
A. Constant	benchmark, regression, homogeneous conditions	vanadis.cfg; keep field routines at defaults
B. Analytic function	known vertical profile, periodic cycle, ramp/pulse	a few lines in the relevant routine
C. Spatial zones	land-use classes, height layers, multiple sources	if/elseif using x,y,z or nr_el
D. Tabulated data	measurements, met data, CFD, K/α maps	read once, interpolate in memory
E. Coupling	dynamic CFD/met fields	adapter/interpolator; do not alter assembly

3. vanadis.cfg - constant baseline values

vanadis.cfg is the natural place for constant baseline values Q, K, P, V1-V3, and ALPHA. The code reads them once into common /cfg/. In v3.1, the default field routines simply return these values. The cfg values can therefore remain useful as fallbacks, reference scales, or model parameters even when the actual field becomes variable.

```
c Baseline behavior (already present in v3.1)
v(1) = v1_cfg
v(2) = v2_cfg
v(3) = v3_cfg

lambda(1) = k_cfg
lambda(2) = k_cfg
lambda(3) = k_cfg

alfa = alpha_cfg
qs = 0.0d0
```

4. Q(t) - time-dependent emission

The current source_term(i_czas,nr_el,q_in) returns kg/s for the specified element. Vanadis then computes that HEX8 volume and converts q_in to qv=q_in/vol. Therefore, for a single element source, provide the total mass-flow rate, not kg/(m³·s). The routine is called automatically at t_n and t_{n+1}; do not manually theta-average Q.

```
c Example: pulse from 300 s through 900 s in element 1083
  q_in = 0.0d0
  if (nr_el .eq. 1083) then
    if (i_czas .ge. 300 .and. i_czas .le. 900)
      & q_in = q_cfg
  endif
```

For multiple source elements, split the total emission with weights f_e that sum to 1. Then q_in(e)=Q_total(t)·f_e. If you need a true Q(x,y,z,t) [kg/(m³·s)] varying inside an element, make a small extension: evaluate a source_field at each Gauss point and integrate q_v(x,y,z,t) directly instead of using q_in/vol.

Q(t) validation: enable the full mass balance and verify that cumulative Cum_Q matches the integral of the prescribed emission.

5. P(S) - concentration-dependent removal

The current `licz_p(c_el,pzanik)` uses the element-average concentration `c_el` [kg/m³] and returns `P` [1/s]. The current function is:

$$P(S) = P_0 + K_2 |S|, \text{ with } K_2 = P_0 / S_{\text{ref}}$$

$$R(S) = P(S) S = P_0 S + K_2 S^2 \text{ (for physical } S \geq 0 \text{)}$$

The current code uses $S_{\text{ref}} = 5 \cdot 10^{-6} \text{ kg/m}^3 = 5 \text{ mg/m}^3$, so $P(S_{\text{ref}}) = 2P_0$. P_0 comes from `p_cfg`. This is a convenient effective model, but S_{ref}/K_2 should be based on calibration, literature, or an explicit sensitivity study, not chosen by the numerics alone.

```
c Constant removal:
  pzanik = p_cfg

c Current nonlinear model:
  s_ref = 5.0d-6
  k2 = p_cfg / s_ref
  s_abs = dabs(c_el)
  pzanik = p_cfg + k2*s_abs
```

If the physics requires $P(S,t)$ or $P(S,x,y,z,t)$, there are two options. The smallest change is to pass time and, for example, the element centroid to `licz_p`. A more accurate option is to evaluate P at each Gauss point when P varies strongly in space or with within-element concentration. The latter is more consistent but more expensive.

Note: $P > 0$ represents removal. $P < 0$ represents production/growth and should occur only if that behavior is intentionally modeled.

6. v(x,y,z,t) - velocity field

`velocity_field(i_czas,x,y,z,v)` is evaluated at physical Gauss-point coordinates. It may return an analytic profile, interpolated measurements, meteorological-model data, or external CFD data. The same routine is used in volume and surface calculations, and Vanadis calls it at both old and new time levels.

```
c Example: simple time-varying multiplier of the baseline field
  real*8 fac, pi
  pi = 4.0d0*datan(1.0d0)
  fac = 1.0d0 + 0.20d0*d sin(2.0d0*pi*dble(i_czas)/3600.0d0)
  v(1) = fac*v1_cfg
  v(2) = fac*v2_cfg
  v(3) = fac*v3_cfg
```

If $\mathbf{v}$ is not divergence-free, the full mass balance contains a DIV term. This is not an error: Vanadis solves the advective form $\mathbf{v} \cdot \nabla S$, so a field with $\nabla \cdot \mathbf{v} \neq 0$ requires this contribution in the integral balance. If $\mathbf{v}$ is a mean RANS/meteorological velocity, unresolved turbulent mixing should be represented consistently in K_{eff} rather than double-counted.

7. $K(x,y,z,t)$ - diffusion/dispersion

`diffusion_field` returns `lambda(1:3)` [m^2/s]. It is currently a diagonal tensor $\text{diag}(K_x, K_y, K_z)$, so directional anisotropy is available without changing assembly. Every component must be ≥ 0 . DR automatically forms the streamwise diffusivity from local K_x, K_y, K_z . $K=0$ is supported as the pure-advection limit.

```
c Example: height-dependent anisotropic effective dispersion
  fac = 1.0d0 + 0.5d0*dmax1(z,0.0d0)/100.0d0
  lambda(1) = k_cfg*fac
  lambda(2) = k_cfg*fac
  lambda(3) = 0.5d0*k_cfg*fac
```

A full tensor with K_{xy} , K_{xz} , and K_{yz} is not represented by `lambda(1:3)`. If it becomes necessary, both the diffusion bilinear form and the DR streamwise projection λ_s must be extended. Do not approximate a physically important full tensor as merely diagonal.

8. $\alpha(x,y,z,t)$ i q_s - Robin condition/deposition

`alpha_field(i_czas,x,y,z,alfa,qs)` is evaluated at the 3×3 Q4 surface Gauss points. In the current configuration the Robin condition is associated with face 3. α has units m/s and q_s has $kg/(m^2 \cdot s)$. Positive α with positive S removes mass from the domain; q_s can represent a prescribed surface flux.

```
c Example: two surface zones
  if (x .lt. 0.0d0) then
    alfa = alpha_cfg
  else
    alfa = 0.5d0*alpha_cfg
  endif
  qs = 0.0d0
```

9. $XX(x,y,z,t)$ - general pattern for new fields

The same pattern can be used for other parameters: humidity, temperature, LAD, chemical reactivity, land-surface class, source parameters, etc. Prefer a small `xx_field(i_czas,x,y,z,xx)` routine called where that quantity enters the equation. Do not move physical input logic into the CUDA kernel or DBCG solver.

```
subroutine xx_field(i_czas,x,y,z,xx)
  implicit real*8 (a-h,o-z)
  integer*4 i_czas
  real*8 x,y,z,xx

c    analytic function OR interpolation from preloaded data
  xx = ...

  return
end
```

10. External files: correct and efficient use

Do not read files inside `velocity_field/diffusion_field/alpha_field`. These routines are called extremely often - for every element, Gauss point, Picard iteration, and both time levels. I/O there would destroy performance.

Read external data once before the time loop into memory. Field routines should perform interpolation only.

For 4-D data, keep at least the two neighboring time slices needed for temporal interpolation. Large datasets can be streamed with a two/four-slice cache instead of loading a full year.

Assembly is OpenMP-parallel. Shared input arrays are fine if they are read-only during assembly. Do not mutate shared state in field routines without synchronization.

Define behavior outside the data domain explicitly: STOP, boundary value, extrapolation, or cfg fallback. Silent out-of-range access is dangerous.

11. Interpolation in time and space

Vanadis passes the requested `i_czas` and physical `x,y,z` to the field routines. If external data are coarser than the simulation step, interpolate inside the data layer. For a time `t` between `t0` and `t1`, linear interpolation is:

$$XX(t) = (1-\theta) XX(t_0) + \theta XX(t_1), \theta = (t-t_0)/(t_1-t_0)$$

In space, use interpolation appropriate to the data: linear in 1-D, bilinear on a map, trilinear on a 3-D grid, or interpolation on an unstructured dataset. Do not interpolate element indices as if they were physical coordinates.

Important for the $\theta=2/3$ scheme: return the field value at the exact requested time. The code itself builds H^n , H^{n+1} , M^n , M^{n+1} , and F^n/F^{n+1} . Do not manually average v , K , α , or Q before passing them to Vanadis.

12. Example meteorological-data adapter

```
c Data are loaded once elsewhere into read-only arrays.
c Here: interpolate only.
  subroutine velocity_field(i_czas,x,y,z,v)
    implicit real*8 (a-h,o-z)
    integer*4 i_czas
    real*8 x,y,z,v(3)

    call interp_met_velocity(dble(i_czas),x,y,z,
&                           v(1),v(2),v(3))

    return
  end
```

The same pattern can be used for `interp_K` and `interp_alpha`. For a field project, keep one adapter layer for the specific data format rather than scattering file-reading logic throughout the solver.

13. How to choose physically meaningful values

Parameter	Preferred data source	Main cautions
v	measurements, meteorological profile, CFD, reanalysis	time/space-scale consistency; no flow into terrain; consistency with K
K	calibration, effective turbulence, literature, atmospheric model	avoid double-counting turbulence; $K \geq 0$
α	deposition/exchange velocity from literature or experiment	surface, species, humidity, vegetation-state dependence
$P(S)$	kinetics, experiment, fitted data	units, sign, concentration range; do not calibrate to one point only
$Q(t)$	emission measurement, dispenser control, material balance	integral of $Q(t)$ must match total emitted mass

For atmospheric applications, consistency of model scale is crucial. A very detailed $v(x,y,z,t)$ combined with an arbitrary constant K is not necessarily better than a simpler but consistently calibrated mean field and K_{eff} . Parameters should represent the same modeling scale.

14. Validation after every input change

- Start from the constant cfg case and reproduce the reference result.
- Enable only one new dependence at a time: $Q(t)$, then perhaps $v(t)$, then $K(x,z)$, etc.
- Check ranges of $P(S)$, minimum/maximum K , α , and velocity.
- Check for NaN/Inf and Jacobian errors.
- Enable `imass_balance=1` and inspect `MASS_BALANCE.TXT`: `Cum_Q`, `Cum_P`, `Cum_alpha`, `Cum_dir`, `Cum_div`, and `Cum_eps`.
- For variable fields, manually verify several interpolation points against the source dataset.
- Record the exact data version, interpolation method, and units.

Minimum acceptance test: when the new dependence is disabled (for example by returning the cfg constant everywhere), the new data layer should reproduce the baseline result to round-off. This is the simplest way to catch adapter/interpolation errors.

15. What to change and what to leave alone

Change	Usually leave unchanged
<code>source_term</code> / optional <code>source_field</code>	DBCG/CUDA just to introduce new physics
<code>licz_p</code> or a new P function	DR when only input data change
<code>velocity_field</code> , <code>diffusion_field</code> , <code>alpha_field</code>	27/9 quadrature for ordinary input changes
external-data adapter/interpolator	global EBE assembly unless necessary
parameters and input files	Dirichlet conditions unless the physical problem changes them

Summary: Vanadis v3.1 already provides explicit hooks for $Q(t)$, $P(S)$, $v(x,y,z,t)$, $K(x,y,z,t)$, and $\alpha(x,y,z,t)$. The safest development strategy is to keep these dependencies in small, testable routines and treat the FEM solver as a stable core. When the physics exceeds the current interface - for example Q varying inside an element, a full K tensor, or strongly spatial $P(S,x,y,z,t)$ - make a small local extension exactly where that quantity enters the FEM integral, then confirm it with regression tests and the full mass balance.